Compilation Project

ABBASSI, BOUZID

March 2026

Partie 7 – A simple stack language

Exercise 1 (expl)

A stack is a data structure following the *LIFO* principle (*Last In, First Out*): the last element pushed onto the stack is the first one to be removed.

The usual operations on a stack are:

- **push**: add an element on the top of the stack.
- **pop**: remove the top element of the stack.
- **top** (or **peek**): read the top element without removing it.
- **is_empty**: test whether the stack is empty.

Exercise 2 (expl)

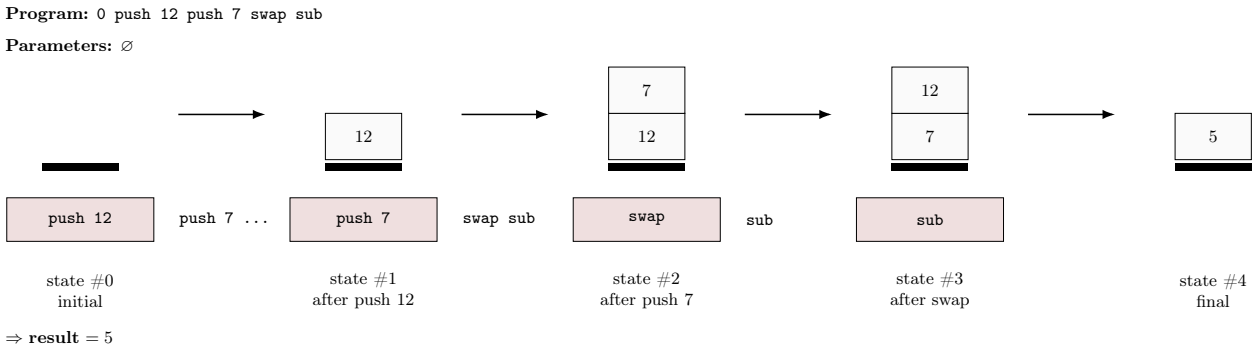

Figure 1: Detailed execution of `0 push 12 push 7 swap sub`.

Exercise 3

Question 3.1 (expl)

The semantics of programs is defined by three rules.

1. If the program expects i arguments but receives n arguments with $i \neq n$, then execution immediately fails and returns `Err`.
2. If the number of arguments is correct, then the arguments are first placed on the initial stack, and the instruction sequence Q is executed. If this execution leads to a runtime error, then the whole program returns `Err`.

3. If the number of arguments is correct and the execution terminates normally on a non-empty stack of the form $v :: S$, then the result of the program is the top value v.

   In short:

- wrong number of arguments $\Rightarrow$ `Err`,
- runtime error during execution $\Rightarrow$ `Err`,
- successful execution with a non-empty final stack $\Rightarrow$ result is the top of the stack.

Question 3.2 (math)

A case is missing: when the number of arguments is correct, execution terminates normally, but the final stack is empty. In that case, there is no result to return, so the program must return `Err`.

The missing rule is:

$$\frac{Q,\ v_1 :: \cdots :: v_n :: \varnothing \to^* \varnothing,\ \varnothing}{v_1, \ldots, v_n \vdash n, Q \Rightarrow \text{Err}}$$

Question 3.3 (math)

We now define the small-step semantics of instruction sequences.

Push

$$\frac{}{\texttt{push}\ k.Q,\ S \to Q,\ k :: S}$$

Pop

$$\frac{}{\texttt{pop}.Q,\ k :: S \to Q,\ S} \qquad \frac{}{\texttt{pop}.Q,\ \varnothing \to \text{Err}}$$

Swap

$$\frac{}{\texttt{swap}.Q,\ k_1 :: k_2 :: S \to Q,\ k_2 :: k_1 :: S}$$

$$\frac{}{\texttt{swap}.Q,\ \varnothing \to \text{Err}} \qquad \frac{}{\texttt{swap}.Q,\ k :: \varnothing \to \text{Err}}$$

Add

$$\frac{}{\texttt{add}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 + k_2) :: S}$$

$$\frac{}{\texttt{add}.Q,\ \varnothing \to \text{Err}} \qquad \frac{}{\texttt{add}.Q,\ k :: \varnothing \to \text{Err}}$$

Sub

$$\frac{}{\texttt{sub}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 - k_2) :: S}$$

$$\frac{}{\texttt{sub}.Q,\ \varnothing \to \text{Err}} \qquad \frac{}{\texttt{sub}.Q,\ k :: \varnothing \to \text{Err}}$$

Mul

$$\frac{}{\texttt{mul}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 \times k_2) :: S}$$

$$\frac{}{\texttt{mul}.Q,\ \varnothing \to \text{Err}} \qquad \frac{}{\texttt{mul}.Q,\ k :: \varnothing \to \text{Err}}$$

Div

$$\frac{k_2 \neq 0}{\texttt{div}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 \div k_2) :: S}$$

$$\frac{}{\texttt{div}.Q,\ \varnothing \to \text{Err}} \qquad \frac{}{\texttt{div}.Q,\ k :: \varnothing \to \text{Err}} \qquad \frac{}{\texttt{div}.Q,\ k_1 :: 0 :: S \to \text{Err}}$$

Rem

$$\frac{k_2 \neq 0}{\texttt{rem}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 \bmod k_2) :: S}$$

$$\overline{\texttt{rem}.Q,\ \varnothing \to \text{Err}} \qquad \overline{\texttt{rem}.Q,\ k :: \varnothing \to \text{Err}} \qquad \overline{\texttt{rem}.Q,\ k_1 :: 0 :: S \to \text{Err}}$$

Final configuration When the instruction sequence is empty, execution is finished:

$$\varnothing,\ S$$

is a final configuration.

Exercise 5

Question 5.1

At this stage, variables cannot be compiled because the current version of *Pfx* does not provide any instruction to access an environment. Therefore, the compilation schema is defined only for variable-free arithmetic expressions.

Instruction sequences are built with the constructor "." and the empty sequence is written $\emptyset$. We define the compilation as:

$$\mathcal{C}(e) = (0, \mathcal{G}(e, \emptyset))$$

where $\mathcal{G}(e, Q)$ means "generate the code that evaluates e and then continues with the instruction sequence Q".

We use the following correspondence between `Expr` binary operators and `Pfx` instructions:

$$\widehat{+} = \texttt{add}, \qquad \widehat{-} = \texttt{sub}, \qquad \widehat{*} = \texttt{mul}, \qquad \widehat{/} = \texttt{div}, \qquad \widehat{\%} = \texttt{rem}.$$

The translation rules are:

$$\mathcal{G}(n, Q) = \texttt{push}\ n.Q$$

$$\mathcal{G}(-e, Q) = \mathcal{G}(e, \texttt{push}\ 0.\texttt{sub}.Q)$$

$$\mathcal{G}(e_1 \oplus e_2, Q) = \mathcal{G}(e_2, \mathcal{G}(e_1, \widehat{\oplus}.Q)) \qquad (\oplus \in \{+, -, *, /, \%\})$$

Variables are not compiled at this stage: $\mathcal{G}(x, Q)$ is undefined because the current *Pfx* machine has no instruction to fetch the value of a variable.

The order in the binary-operation rule is important. In *Pfx*, arithmetic instructions use the top element of the stack as their first argument and the second element as their second argument. Therefore, to compile $e_1 - e_2$, we must first push the value of e_2 and then the value of e_1, so that `sub` computes $e_1 - e_2$.

The intended correctness property is the following: if e is a variable-free expression whose value is v, then executing $\mathcal{G}(e, Q)$ on a stack S has the same effect as executing Q on the stack $v :: S$. In particular, executing $\mathcal{C}(e)$ leaves the value of e on top of the initial empty stack.

Exercise 9

Question 9.1

We do not need to *change the behavior* of the already defined *Pfx* instructions, but we do need to enlarge the domain on which they operate.

Previously, the stack contained only integers. After the introduction of executable sequences, a stack element becomes either:

$$v ::= n \mid \langle Q \rangle$$

where $n \in \mathbb{Z}$ and Q is an instruction sequence.

Therefore:

- the rules for `push`, `pop`, and `swap` keep the same operational behavior; they now manipulate stack *objects* instead of integers only;
- the arithmetic instructions `add`, `sub`, `mul`, `div`, and `rem` keep exactly the same computation rule, but they are only defined when their operands are integers;
- if one of these arithmetic instructions is applied to an executable sequence, the machine should raise a runtime error.

So the right answer is: the old rules are essentially preserved, but the type of stack elements must be generalized from integers to stack values $v \in \mathbb{Z} \cup \{\langle Q \rangle\}$.

Question 9.2

We give the operational semantics of the three new *Pfx* constructions: executable sequences, `exec`, and `get`.

We write machine states as pairs (Q, S) where:

- Q is the currently executing instruction sequence;
- S is the stack;
- instruction sequences are written with the paper notation $I.Q$ and $\emptyset$.

When a whole sequence must be put in front of another one, we still use the same dot notation recursively:

$$\emptyset.Q = Q \qquad \text{and} \qquad (I.Q_1).Q_2 = I.(Q_1.Q_2)$$

Executable sequence. When an executable sequence is encountered in the current code, it is not executed immediately. It is pushed onto the stack as a value.

$$\overline{(\langle Q_1 \rangle.Q_2,\ S) \ \longrightarrow\ (Q_2,\ \langle Q_1 \rangle :: S)}$$

`exec`. The instruction `exec` pops the top of the stack. This top element must be an executable sequence. The popped code is then executed immediately by being placed in front of the remaining current code.

$$\overline{(\texttt{exec}.Q,\ \langle Q' \rangle :: S) \ \longrightarrow\ (Q'.Q,\ S)}$$

If the top element of the stack is not an executable sequence, the machine raises a runtime error.

`get`. The instruction `get` pops an integer i from the top of the stack, then copies onto the top of the stack the element of depth i in the remaining stack. The indexing is 0-based: depth 0 means the top of the remaining stack.

$$\frac{i \geq 0}{(\texttt{get}.Q,\ i :: v_0 :: v_1 :: \cdots :: v_i :: S) \ \longrightarrow\ (Q,\ v_i :: v_0 :: v_1 :: \cdots :: v_i :: S)}$$

In all other cases, `get` raises a runtime error:

- if the top of the stack is not an integer;
- if the stack does not contain enough elements below this integer.

Remark. These rules are exactly what is needed for compiling simple functions:

- a function is translated into an executable sequence;
- an application uses `exec` to launch this sequence;
- the instruction `get` gives access to a parameter stored deeper in the stack.

Exercise 10

Question 10.1

We want to compile the expression

$$(\lambda x.\ x + 1)\ 2$$

into *Pfx*.

Using the translation principle of the statement:

- a function is translated into an executable sequence;
- an application first computes the argument, then the function value, then runs `exec`;
- after the call terminates, the argument is removed from the stack with `swap.pop`.

The compiled program is therefore:

$$(0,\ \texttt{push 2.(push 1.push 1.get.add.}\emptyset\texttt{).exec.swap.pop.}\emptyset)$$

Why is the function body compiled this way? Inside the body $x+1$, the constant 1 is computed first. Once this value is pushed onto the stack, the parameter x is no longer at depth 0 but at depth 1. Therefore, the occurrence of x is compiled as:

$$\texttt{push 1.get}$$

Hence:

$$\lambda x.\ x + 1 \quad \rightsquigarrow \quad (\texttt{push 1.push 1.get.add.}\emptyset)$$

Step-by-step evaluation. We start from the empty stack.

$$(\texttt{push 2.}(Q_f)\texttt{.exec.swap.pop.}\emptyset,\ [])$$

where

$$Q_f = \texttt{push 1.push 1.get.add.}\emptyset$$

$$\begin{aligned}
(\texttt{push 2.}(Q_f)\texttt{.exec.swap.pop.}\emptyset,\ []) &\to ((Q_f)\texttt{.exec.swap.pop.}\emptyset,\ [2]) \\
&\to (\texttt{exec.swap.pop.}\emptyset,\ [Q_f; 2]) \\
&\to (Q_f\texttt{.swap.pop.}\emptyset,\ [2]) \\
&\to (\texttt{push 1.get.add.swap.pop.}\emptyset,\ [1; 2]) \\
&\to (\texttt{get.add.swap.pop.}\emptyset,\ [1; 1; 2]) \\
&\to (\texttt{add.swap.pop.}\emptyset,\ [2; 1; 2]) \\
&\to (\texttt{swap.pop.}\emptyset,\ [3; 2]) \\
&\to (\texttt{pop.}\emptyset,\ [2; 3]) \\
&\to (\emptyset,\ [3])
\end{aligned}$$

So the final result is:

$$(\lambda x.\ x + 1)\ 2 = 3$$

This exactly matches Figure 1 of the statement:

- first the argument 2 is pushed;
- then the executable sequence representing the function is pushed;
- `exec` starts the call with the argument still on the stack;
- the body computes the result while the parameter remains accessible;
- finally `swap.pop` removes the parameter and keeps only the result.

Question 10.2

We define a compilation function with an environment P that associates each accessible variable with its depth in the current stack.

$$\mathcal{G}_P(e, Q)$$

means: compile the expression e in environment P, assuming that after evaluation the generated code continues with instruction sequence Q.

Auxiliary definitions. When a value is pushed on the stack, all accessible variables become one position deeper. We write:

$$P^{+1}(x) = P(x) + 1$$

When introducing a new parameter y at the top of the stack, we extend the environment by:

$$P[y](x) = \begin{cases} 0 & \text{if } x = y, \\ P(x) + 1 & \text{if } x \neq y. \end{cases}$$

We also use the correspondence:

$$\widehat{+} = \texttt{add}, \quad \widehat{-} = \texttt{sub}, \quad \widehat{*} = \texttt{mul}, \quad \widehat{/} = \texttt{div}, \quad \widehat{\%} = \texttt{rem}.$$

Compilation rules.

$$\mathcal{G}_P(n, Q) = \texttt{push}\ n.Q$$

$$\mathcal{G}_P(x, Q) = \texttt{push}\ P(x).\texttt{get}.Q$$

$$\mathcal{G}_P(-e, Q) = \mathcal{G}_P(e, \texttt{push}\ 0.\texttt{sub}.Q)$$

$$\mathcal{G}_P(e_1 \oplus e_2, Q) = \mathcal{G}_P(e_2, \mathcal{G}_{P^{+1}}(e_1, \widehat{\oplus}.Q)) \qquad (\oplus \in \{+, -, *, /, \%\})$$

$$\mathcal{G}_P(\lambda x.\ e, Q) = \left(\mathcal{G}_{P[x]}(e, \emptyset)\right).Q$$

$$\mathcal{G}_P(e_1\ e_2, Q) = \mathcal{G}_P(e_2, \mathcal{G}_{P^{+1}}(e_1, \texttt{exec.swap.pop}.Q))$$

Explanation of the application rule. The expression e_2 is compiled first, so its value is pushed onto the stack. Therefore, while compiling e_1, all previously accessible variables are one level deeper, which explains the use of P^{+1}.

After both subexpressions have been evaluated, the stack has the form:

[function code; argument; rest of the stack]

Then:

- `exec` starts the executable sequence;
- the function body computes a result while keeping the argument on the stack;
- `swap.pop` removes the argument and leaves only the result.